Statistical and Computational Models

Lecture 18: Neural networks

Daniel de Kadt

Department of Government, Cornell University · Fall 2026

Roadmap

Learned basis functions

Choosing the knots

Many features

Interpreting neural networks

Our classification problem

Last time we gave the logit flexibility with **basis functions**:

$$\text{logit}\,\pi(\boldsymbol{x}) = \beta_0 + \sum_{j=1}^{J} \beta_j\, b_j(\boldsymbol{x})$$

Splines and GAMs fix the b_j before fitting. The knots go at quantiles of $\boldsymbol{x}$, CV selects how many, and the data give us the $\hat{\beta}$s via (some variety of) MLE.

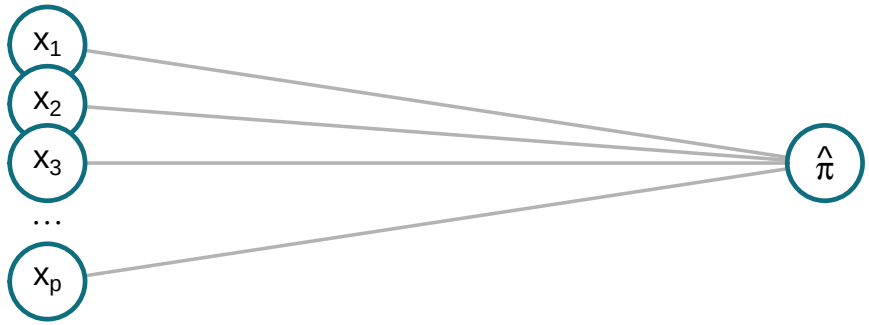
Now we let the data choose the b_j too. This is a **neural network**.

Same two goals:

1. Rank this election's stations.
2. Learn how the chance of an irregularity changes with each feature.

A logit as a network

Draw the logit as a **network**: input features on the left, $\hat{\pi}$ on the right:



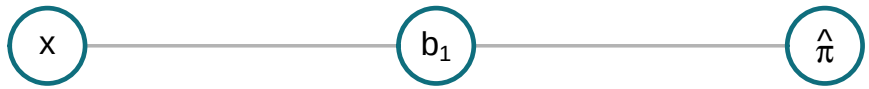
$$\text{logit}\, \pi(\boldsymbol{x}) = \beta_0 + \sum_{j=1}^{p} \beta_j x_j$$

This is a directed acyclic graph (DAG), though not a causal one:

1. Nodes: each input feature, and the output.
2. Edges: one **weight** each, β_j from $\boldsymbol{x}_j$ to $\hat{\pi}$.
3. We elide arrows, as we will strictly read from left to right.

A hidden layer

Draw the logit as a **network**: input features on the left, $\hat{\pi}$ on the right:

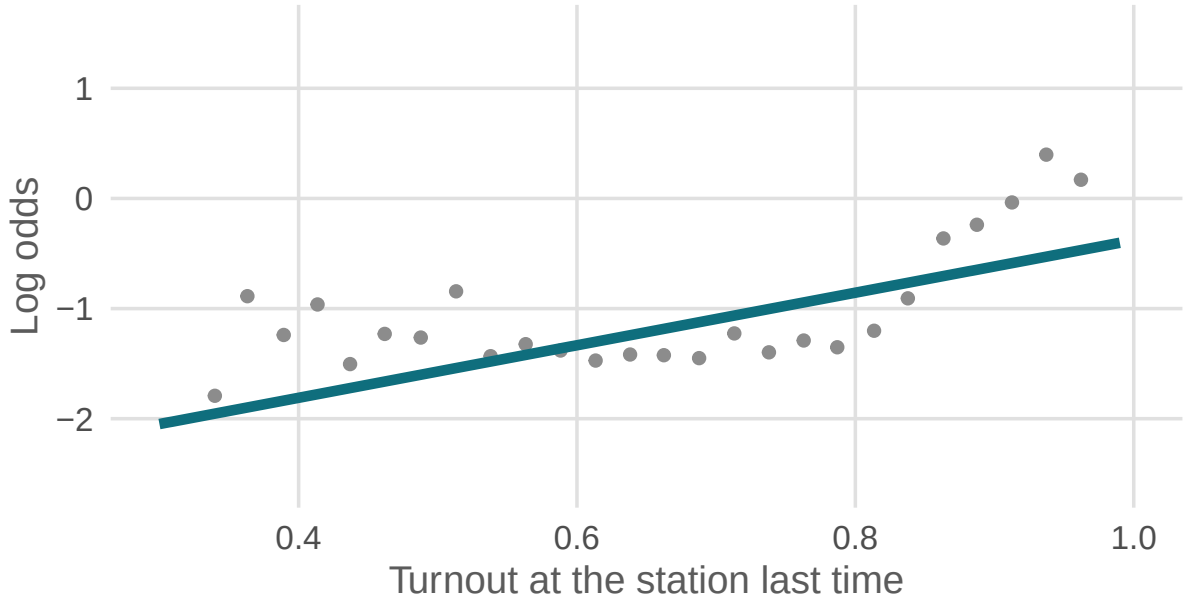


$$\text{logit}\,\pi(x) = \beta_0 + \beta_1 b_1(x)$$

To make the model more flexible, we put a **hidden layer** of basis functions – what we will call **derived features** (aka hidden units, neurons) – between the inputs and the output. Unlike the spline, the output sees only the derived features, not the inputs.

To see how we get there, let's focus again on just one feature – turnout. To begin with, we'll learn how to generate a single derived feature for turnout – a hidden layer with only one derived feature, b_1.

Logit with a single feature

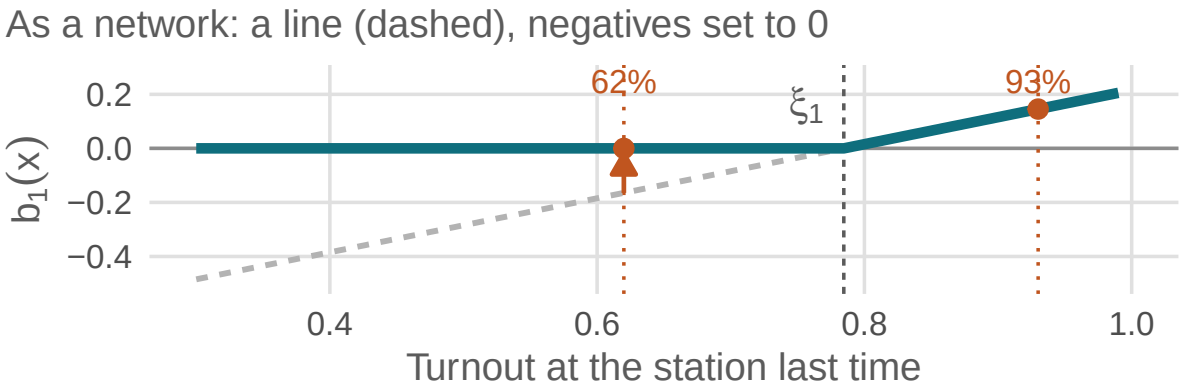


With x as its only column, the logit is a straight line in log odds space: $-2.76 + 2.39\,x$.

So, as with a spline, we build a new column from x: piecewise linear, with a knot. This lets the relationship between x and Y be flexible.

One derived feature

Pick a knot, $\xi_1 = 0.785$ (we will come back to how this happens shortly). The new column is 0 until x passes it, then a positive straight line:



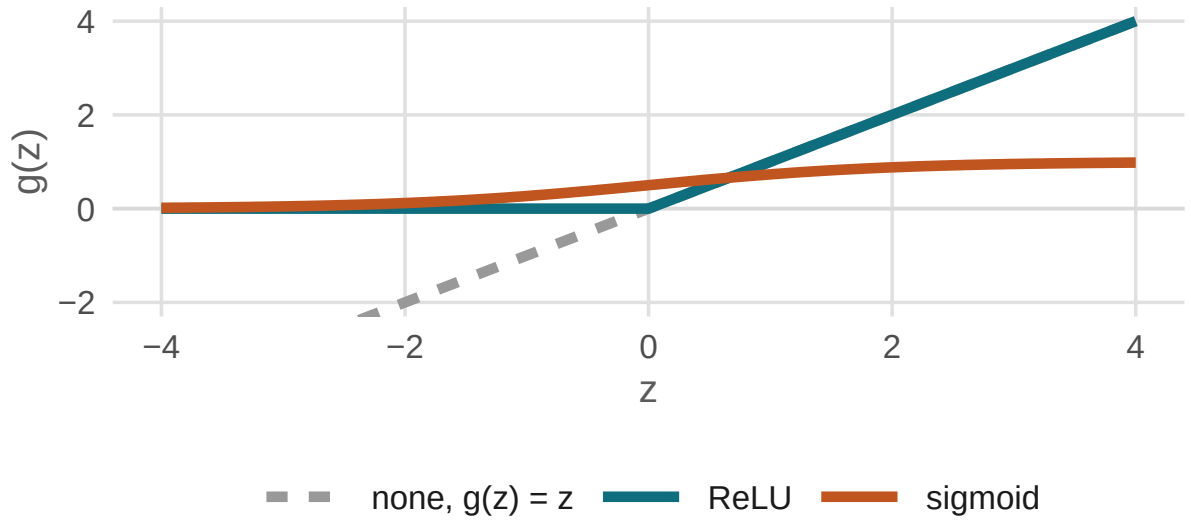
$$b_1(x) = (x - 0.785)_+$$
$$= \max(0,\ -0.785 + x) = \max(0,\ w_{10} + w_{11}\, x)$$

w_{10} is the derived feature's intercept (aka **bias**, not the bias–variance kind), which sets the knot: $\xi_1 = -w_{10}/w_{11}$. For the ReLU, the magnitude of w_{11} is arbitrary, so for exposition we set $w_{11} = 1$. So $w_{10} = -0.785$.

This is our first **derived feature**.

The activation

$$z_j = w_{j0} + w_{j1} x \text{ (a line)}, \qquad b_j = g(z_j) \text{ (a bend)}$$

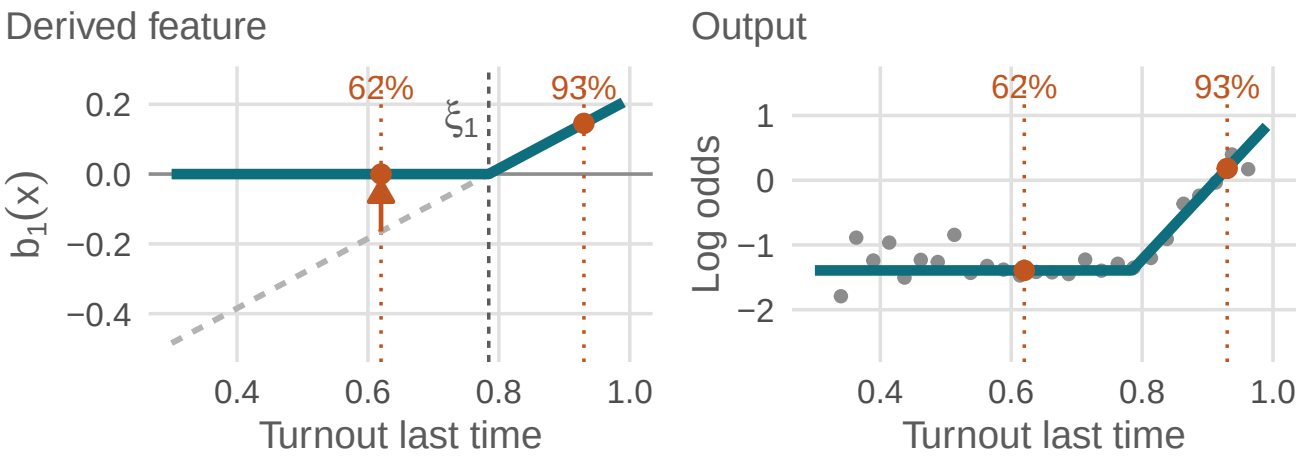


The shape of the basis function is called the **activation function**, $g(\cdot)$.
Without a bend, $g(z) = z$: b_j is a line, and the model is just the logit again.

Ours is the **ReLU** (rectified linear unit): $g(z) = \max(0, z)$.

There are other options. The **sigmoid**, $\text{logit}^{-1}(z)$, gives a smooth bend.

A logit on the derived feature



$$\text{logit}\,\hat{\pi}(x) = \underbrace{-1.39}_{\beta_0} + \underbrace{10.83}_{\beta_1}\, b_1(x)$$

Read: β_0 is the log odds below the knot, and β_1 the slope after it.

62%: $\text{logit}\,\hat{\pi} = -1.39 + 10.83 \times 0 = -1.39 \rightsquigarrow \hat{\pi} = 0.199$.
93%: $\text{logit}\,\hat{\pi} = -1.39 + 10.83 \times 0.145 = 0.18 \rightsquigarrow \hat{\pi} = 0.545$.

So the shape of the output follows from the shape of the activation.

Roadmap

Learned basis functions

Choosing the knots

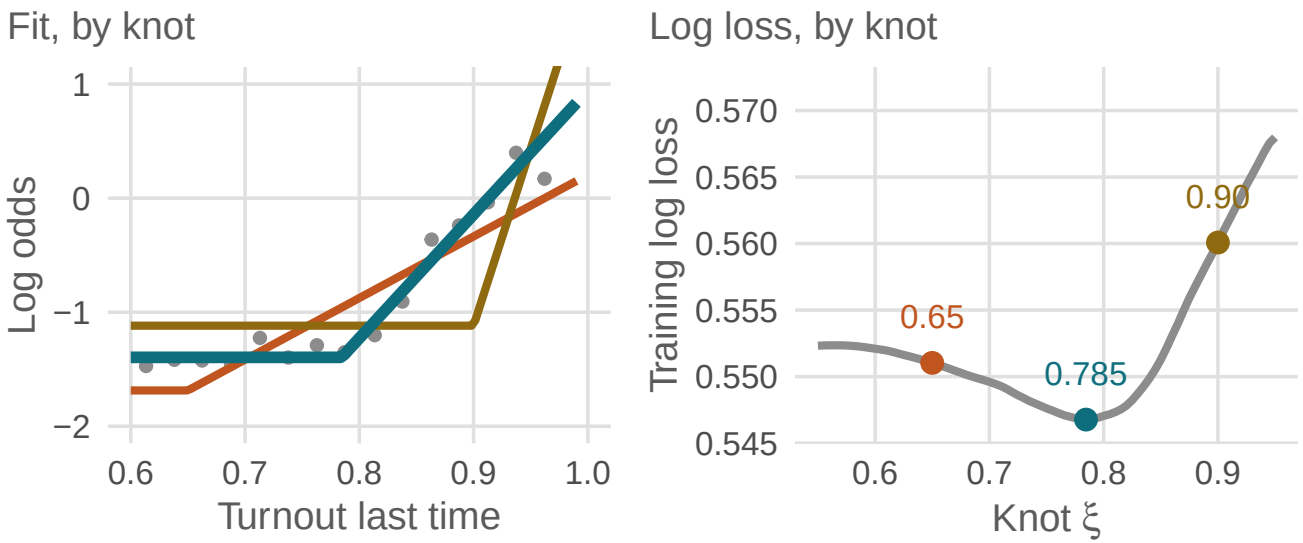
Many features

Interpreting neural networks

Choosing the knot

Try a knot ξ, refit the βs, and record the log loss:

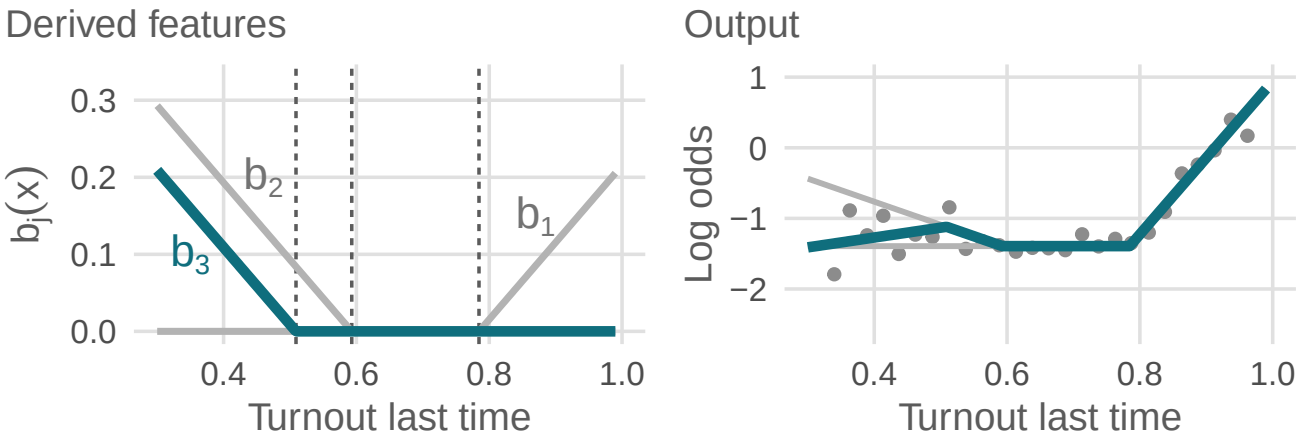
$$\text{log loss} = -\frac{1}{n}\sum_{i=1}^{n}\left\{y_i \log \hat{\pi}_i + (1 - y_i)\log(1 - \hat{\pi}_i)\right\}$$



ξ = 0.65: too early. ξ = 0.90: too late. ξ = 0.785: just right!

Multiple derived features

But a single knot only lets the fit bend once. So let's add more derived features, each with its own knot:



$$\text{logit}\,\hat{\pi}(x) = -1.39 + 10.83\,b_1(x) + 3.26\,b_2(x) - 4.66\,b_3(x)$$

A negative w_{j1} flips the ramp so that it is 0 above the knot.

So the output's shape is now a weighted sum of their shapes.

The network's design matrix

The **design matrix** of the output layer, for four of our stations:

Y	x	1	$b_1(x)$	$b_2(x)$	$b_3(x)$	$\hat{\pi}$
0	0.450	1	0	0.143	0.059	0.23
1	0.620	1	0	0	0	0.20
0	0.780	1	0	0	0	0.20
1	0.930	1	0.145	0	0	0.55
⋮	⋮	⋮	⋮	⋮	⋮	⋮

A **forward pass** runs each row left to right: $x_i \to b_j(x_i) \to \text{logit}\,\hat{\pi}_i \to \hat{\pi}_i$.

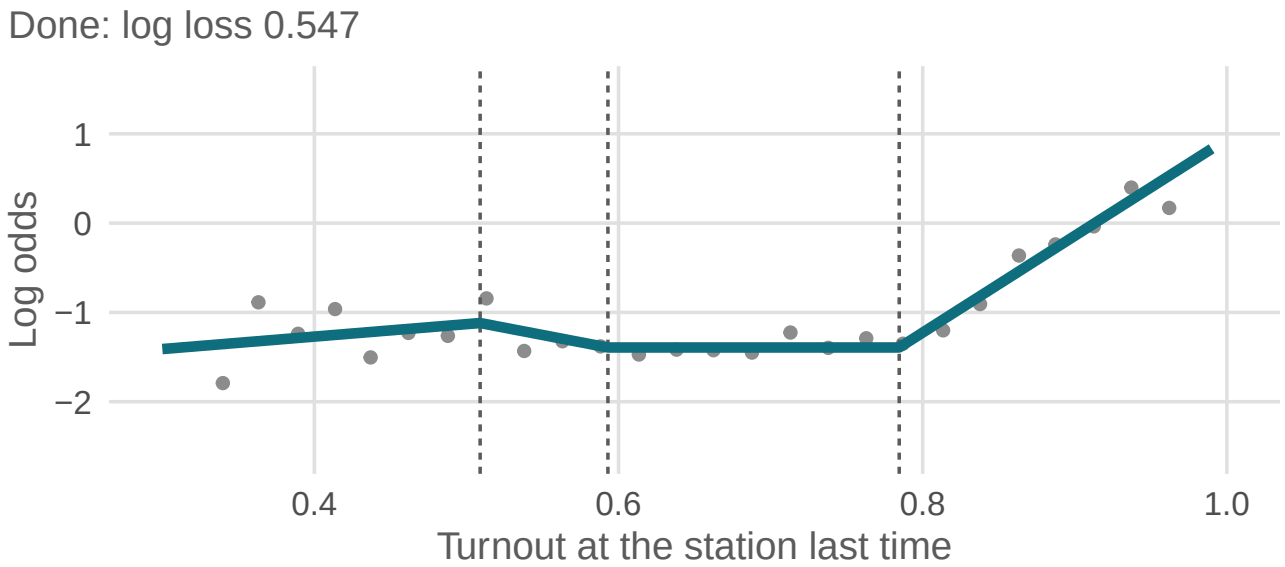
Row 4: $\text{logit}\,\hat{\pi} = -1.39 + 10.83 \times 0.145 = 0.18 \rightsquigarrow \hat{\pi} = 0.545$.

Note: no x column (grey), unlike the spline.

A spline's columns are fixed before fitting, so `glm()` can fit it. Ours depend on the knots, which we solve for at the same time as the βs.

Fitting many knots

With a single knot, we could try all possible options. With three knots and their βs, there are too many combinations. So we minimize the log loss over all 10 weights:

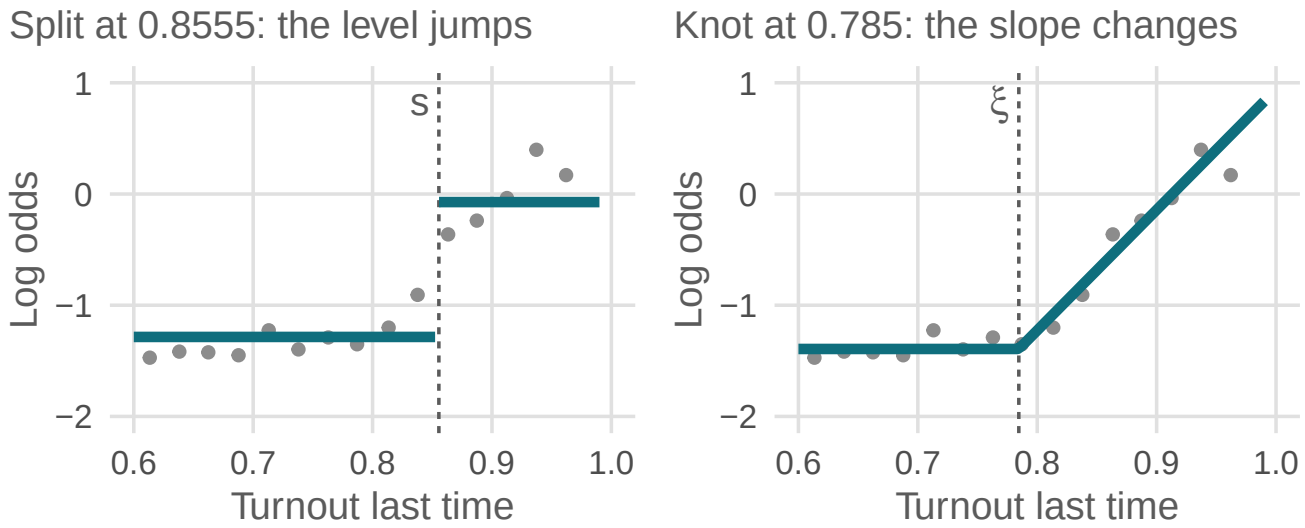


Start from a random guess of three knots (dashed lines). Then take a Newton-type step, using the slope of the log loss and an estimate of its curvature (`optim()`'s quasi-Newton, as for the MLE), and repeat.

Different starting guesses can end in different places. So we try 30 random starts, and keep the fit with the lowest log loss.

Aside: trees and networks

A tree's split is a derived feature too, with a step as its activation:



split: $b(x) = 1\{x - 0.8555 \geq 0\}$, knot: $b(x) = \max(0, x - 0.785)$

A tree adds one split at a time and never revisits it. A network fits all its knots together.

Roadmap

Learned basis functions

Choosing the knots

Many features

Interpreting neural networks

Six features

With many features, each b_j takes a weighted mix of every input:

$$b_j(\boldsymbol{x}) = g\left(w_{j0} + \sum_{k=1}^{p} w_{jk} x_k\right), \qquad \text{logit}\,\pi(\boldsymbol{x}) = \beta_0 + \sum_{j=1}^{J} \beta_j\, b_j(\boldsymbol{x})$$

Read: each b_j bends where its own mix of the features passes zero.

So the network explores **interactions**, as a tree does. Note that `nnet` uses the sigmoid for g, so $\hat{\pi}$ stays smooth, as in a GAM.

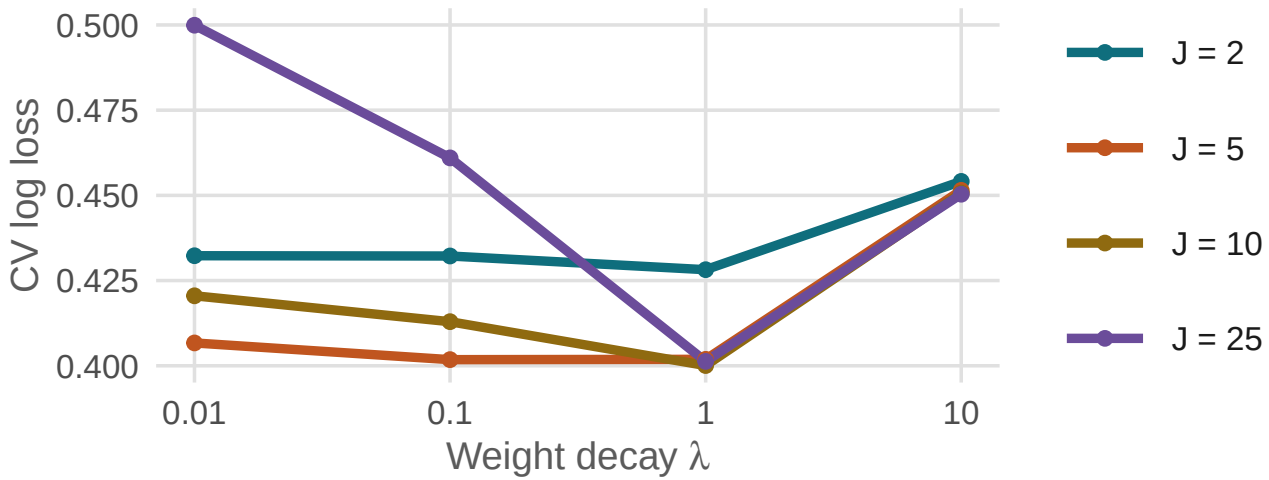
How many weights? For $p = 11$ inputs and $J = 10$ derived features: $(p + 1)J + J + 1 = 131$.

We fit them as we fit turnout: `nnet`, from ten random starts, keeping the best.

Tuning the network

More derived features, more flexibility, and more room to fit noise – **overfitting**. We need to **tune** the network, with two parameters:

1. the **width** J: how many derived features.
2. **weight decay** λ: similar to ridge, minimize
   $n \times$ log loss $+ \lambda \times$ (sum of every squared weight).



By CV, on the spline's ten folds: $J = 10$, $\lambda = 1$.

In R

`nnet()` fits one hidden layer:

```
library(nnet)
set.seed(14850)                          # nnet starts from random weights
Z <- scale(X_tr)                         # standardize: mean 0, sd 1
fit_nn <- nnet(Z, train$y,
               size = 10,                # J: derived features
               decay = 1,                # lambda: weight decay
               entropy = TRUE,           # log loss (default: squared error)
               maxit = 500)              # at most 500 quasi-Newton steps
Z_te <- scale(X_te, attr(Z, "scaled:center"),   # test set: training
                    attr(Z, "scaled:scale"))    # means and sds
p_nn <- predict(fit_nn, Z_te)            # pi-hat for each station
```

Standardize the inputs first! As with ridge, the penalty is not scale invariant (one part of 'feature engineering').

Roadmap

Learned basis functions

Choosing the knots

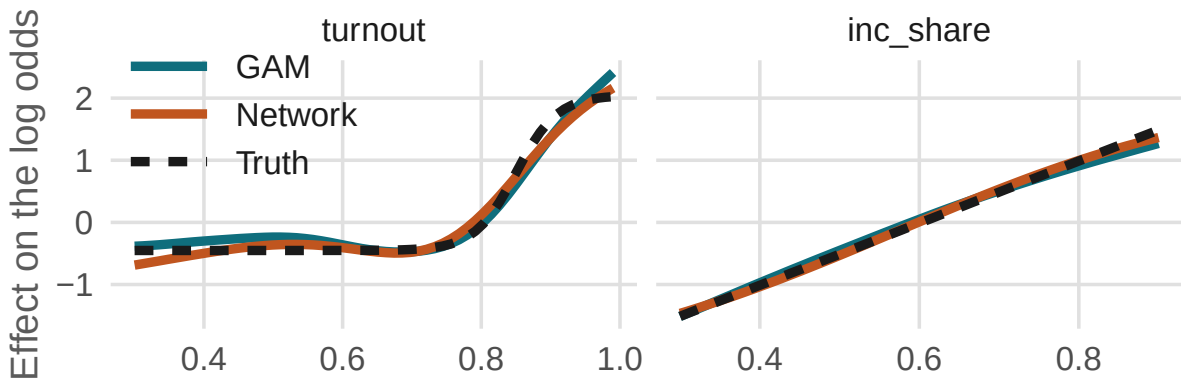
Many features

Interpreting neural networks

Interpreting a network

No $\hat{f}_j$ to plot, so we compute one, **partial dependence**:

1. Set every station's turnout to t.
2. Predict, and average logit $\hat{\pi}$.
3. Repeat over t.



Dashed: the truth.

Interpreting a network

Does a missing agent matter less in urban stations? In truth, yes: −1.0 on the log odds. Partial dependence on both features, as a difference in differences:

Model	urban × no agent
Truth	−1.00
GAM	0.00
Network	−0.64

The GAM will find no interaction unless we specified it in the model. The network finds some of it without our intervention.

Evaluating our candidate classifiers

Model	Test AUC	Turnout effect
Logit	0.825	0.087
Spline logit	0.838	0.349
GAM	0.838	0.344
Random forest	0.858	0.290
Boosting	0.858	0.350
Network	0.863	0.279
Truth	0.869	0.306

On this election's stations. Turnout effect: the average change in $\hat{\pi}$ from 80% to 95% turnout.

The network ranks with the forest and boosting, and its turnout effect is within 3 points of the truth.

But the gains are small: with six features, boosting is nearly as good, and easier to tune. So why bother?

When do networks win?

1. **Tabular** data, a few meaningful features: boosting is hard to beat, and cheaper to tune. The GAM is easier to read. Most political science data is like this: low-dimensional, so complex models gain little (Morucci and Spirling).
2. **Raw**, high-dimensional inputs with structure (pixels, words, audio): we cannot hand-build the basis functions.

But millions of weights need something cheaper than quasi-Newton steps to fit (next time).

References

- James, G., Witten, D., Hastie, T. & Tibshirani, R. (2021) *An Introduction to Statistical Learning*, 2nd ed. – **10.1, 10.6**. statlearning.com.